

ermediate-leadership-competency.md" %}}

{{% include "includes/engineering-careers/development-intermediate-leadership-competency.md" %}}

Distribution

- Participates in team trainings, issue triage, and process review.
- Performs Code Reviews within their domain on a regular cadence.
- Intentionally develops the skills necessary to become an expert on a specific subject.
- Shares knowledge with team members after becoming a domain expert.
- Seeks and reviews customer issues and feedback through joining customer support calls, and reading customer feedback.

Technical Competencies

{{% include "includes/engineering-careers/intermediate-technical-competency.md" %}}

{{% include "includes/engineering-careers/development-intermediate-technical-competency.md" %}}

Distribution

- Capable of presenting training or demos, within their team domain.
- Proficient or fluent with one or more languages to build, deploy and manage GitLab installations.
- Working knowledge of Distribution's build and installation frameworks.
- Able to debug issues that occur in production during installation and upgrading.
- Develops documentation and instructions explaining how to configure, install and manage GitLab service components.
- Able to install and review GitLab installations in multiple deployment configurations.

Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

{{% include "includes/engineering-careers/development-intermediate-values-competency.md" %}}

SECTION: engineering/careers/matrix/infrastructure/core-platform/distribution/senior.md

Core Platform Senior Distribution Engineer

{{% include "includes/engineering-careers/distribution-career-nav.md" %}}

Core Platform Senior Distribution Engineers at GitLab are expected to exhibit the following competencies:

- Leadership Competencies
- Technical Competencies

- Values Alignment
- All competencies exhibited by an Intermediate

Leadership Competencies

{{% include "includes/engineering-careers/senior-leadership-competency.md" %}}

{{% include "includes/engineering/development-senior-leadership-competency.md" %}}

Distribution

- Presents training or demos within their domain on a regular cadence.
- Leads technical architecture discussions and help drive technical decisions.
- Works with project maintainers, and contributes to maintainer discussion issues.
- Participates on Working Groups as the need arises.
- Participates on customer support/feedback calls as the need arises.
- Act as a coach for releasing service components to end-users.
- Able to own a engineering project from concept to production, including proposal, discussion, and execution.

Technical Competencies

{{% include "includes/engineering-careers/senior-technical-competency.md" %}}

{{% include "includes/engineering-careers/development-senior-technical-competency.md" %}}

Distribution

- Provides high level of expertise in programming languages and their frameworks used in Distribution managed engineering projects.
- Acts as a subject matter expert in one or more linux system deployment technology.
- Able to debug production issues across services and identify the product category.
- Able to provide deployment recommendations for production installations.
- Understand how features in their domain operate at scale in production.

Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

{{% include "includes/engineering-careers/development-senior-values-competency.md" %}}

Intermediate Competencies

All competencies exhibited by an Intermediate

SECTION: engineering/careers/matrix/quality/_index.md

Quality Department Career Framework

{{% include "includes/engineering-careers/quality-career-nav.md" %}}

Here is a flowchart representation of the Career development in Quality Department.

Software Engineer in Test

Quality Sub-departments

- Dev
- Ops
- Sec
- Core Platform
- SaaS Platforms
- Growth
- Fulfillment
- Data Science

Career Framework

These are the expected competencies of the Quality Software Engineer in Test team members by career level.

1. Associate
1. Intermediate
1. Senior
1. Staff
1. Principal

Backend Engineer, Engineering Productivity

These are the expected competencies of the Quality Backend Engineer, Engineering Productivity role by career level.

1. Intermediate
1. Senior
1. Staff

SECTION: engineering/careers/matrix/quality/engineering-productivity/_index.md

Engineering Productivity Career Framework

{{% include "includes/engineering-careers/ep-career-nav.md" %}}

These are the expected competencies of the Backend Engineer, Engineering Productivity team members at GitLab.

Intermediate

- Backend Engineer

Senior

- Backend Engineer

Staff

- Backend Engineer

SECTION: engineering/careers/matrix/quality/engineering-productivity/intermediate.md

Intermediate Backend Engineer, Engineering Productivity

{{% include "includes/engineering-careers/ep-career-nav.md" %}}

Intermediate Backend Engineers, Engineering Productivity at GitLab are expected to exhibit the following competencies:

<!-- markdownlint-disable MD051 -->

- Leadership Competencies
- Technical Competencies
- Values Alignment

<!-- markdownlint-enable MD051 -->

Intermediate Leadership Competencies

{{% include "includes/engineering-careers/intermediate-leadership-competency.md" %}}

Intermediate Technical Competencies

{{% include "includes/engineering-careers/intermediate-technical-competency.md" %}}

Engineering Productivity

This table refers to the responsibility terms that are defined on the team page.

| Responsibility | Description |

| --- | --- |

| See it and find it | Identify opportunities to shorten developer feedback loops and implement improvements for product groups within a stage |

| Do it for internal team | Implement and collaborate with other EP engineers on productivity improvements that deliver quantifiable benefits for GitLab product groups within a stage |

| Dogfood use | Identify existing product features to dogfood or new product improvements that

could be leveraged for GitLab engineering workflows |
Engineering support	Identify KPI corrective actions and improvements. Implement corrective action improvements with guidance.
Engineering workflow	Implement automated processes for improving label classification hygiene in support of product and engineering workflows
Dogfood build	Provides input on enhancements to existing features from team perspective

Intermediate Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

SECTION: engineering/careers/matrix/quality/engineering-productivity/senior.md

Senior Backend Engineer, Engineering Productivity

{{% include "includes/engineering-careers/ep-career-nav.md" %}}

Senior Backend Engineers, Engineering Productivity at GitLab are expected to exhibit the following competencies:

<!-- markdownlint-disable MD051 -->

- Leadership Competencies
- Technical Competencies
- Values Alignment

<!-- markdownlint-enable MD051 -->

Senior Leadership Competencies

{{% include "includes/engineering-careers/senior-leadership-competency.md" %}}

Senior Technical Competencies

{{% include "includes/engineering-careers/senior-technical-competency.md" %}}

Engineering Productivity

This table refers to the responsibility terms that are defined on the team page.

| Responsibility | Description |

| --- | --- |

| See it and find it | Identify and implement changes to shorten developer feedback loops and implement improvements for product groups within multiple stages |

| Do it for internal team | Implement productivity improvements with quantifiable benefits for GitLab product groups within multiple stages |

| Dogfood use | Dogfood existing product features to improve the GitLab engineering workflows. Provide feedback to product teams on critical improvements which will improve the feature

maturity. |
Engineering support	Takes initiative in implementing KPI corrective actions and improvements
Engineering workflow	Collaborate with product and engineering stakeholders to identify and implement improvements or new automation in support of product and engineering workflows
Do it for wider community	Identifies contributor tooling improvements for the wider GitLab community contributors
Dogfood build	Implement small enhancements to existing features

Senior Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

SECTION: engineering/careers/matrix/quality/engineering-productivity/staff.md

Staff Backend Engineer, Engineering Productivity

{{% include "includes/engineering-careers/ep-career-nav.md" %}}

Staff Backend Engineers, Engineering Productivity at GitLab are expected to exhibit the following competencies:

<!-- markdownlint-disable MD051 -->
- Leadership Competencies
- Technical Competencies
- Values Alignment
<!-- markdownlint-enable MD051 -->

Staff Leadership Competencies

{{% include "includes/engineering-careers/staff-leadership-competency.md" %}}

Staff Technical Competencies

{{% include "includes/engineering-careers/staff-technical-competency.md" %}}

Engineering Productivity

This table refers to the responsibility terms that are defined on the team page.

Responsibility Description
--- ---
See it and find it Identify and implement changes to shorten developer feedback loops and implement improvements for product stages within a section
Do it for internal team Implement productivity improvements with quantifiable benefits for GitLab product stages within a section

Dogfood use	Dogfood existing product features to improve the GitLab engineering workflows. Represent the Quality department as internal customer for at least one product group.
Engineering support	Implement KPI corrective actions and preventative measures. Identify areas for new department performance indicators.
Engineering workflow	Lead collaboration and implementation with product and engineering stakeholders to improve automation in support of product and engineering workflows
Do it for wider community	Identifies contributor productivity improvements for the wider GitLab community contributors
Dogfood build	Large enhancements to existing features or addition of new features. Your implementation has a large impact to a product category

Staff Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

SECTION: engineering/careers/matrix/quality/software-engineer-in-test/_index.md

Software Engineer in Test Career Framework

{{% include "includes/engineering-careers/quality-career-nav.md" %}}

These are the expected competencies of the Software Engineer in Test team members at GitLab.

Associate

- Software Engineer in Test

Intermediate

- Software Engineer in Test

Senior

- Software Engineer in Test

Staff

- Software Engineer in Test

Principal

- Software Engineer in Test

SECTION: engineering/careers/matrix/quality/software-engineer-in-test/associate.md

Quality Department Competencies: Associate

{{% include "includes/engineering-careers/quality-career-nav.md" %}}

Associates at GitLab are expected to exhibit the following competencies:

- Associate Leadership Competencies
- Associate Technical Competencies
- Associate Values Alignment

Associate Leadership Competencies

{{% include "includes/engineering-careers/associate-leadership-competency.md" %}}

Associate Technical Competencies

{{% include "includes/engineering-careers/quality-associate-technical-competency.md" %}}

Associate Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

SECTION: engineering/careers/matrix/quality/software-engineer-in-test/intermediate.md

Quality Department Competencies: Intermediate

{{% include "includes/engineering-careers/quality-career-nav.md" %}}

Intermediates at GitLab are expected to exhibit the following competencies:

- Intermediate Leadership Competencies
- Intermediate Technical Competencies
- Intermediate Values Alignment

Intermediate Leadership Competencies

{{% include "includes/engineering-careers/intermediate-leadership-competency.md" %}}

Intermediate Technical Competencies

{{% include "includes/engineering-careers/quality-intermediate-technical-competency.md" %}}

Intermediate Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

Intermediate Career Matrix Worksheet

Template for Intermediate Career Matrix Worksheet can be found [here](#).

SECTION: engineering/careers/matrix/quality/software-engineer-in-test/principal.md

Quality Department Competencies: Principal

{{% include "includes/engineering-careers/quality-career-nav.md" %}}

Principal at GitLab are expected to exhibit the following competencies:

- Principal Leadership Competencies
- Principal Technical Competencies
- Principal Values Alignment

Principal Leadership Competencies

{{% include "includes/engineering-careers/principal-leadership-competency.md" %}}

Principal Technical Competencies

{{% include "includes/engineering-careers/quality-principal-technical-competency.md" %}}

Principal Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

Principal Career Matrix Worksheet

Template for Principal Career Matrix Worksheet - TBD

SECTION: engineering/careers/matrix/quality/software-engineer-in-test/senior.md

Quality Department Competencies: Senior

{{% include "includes/engineering-careers/quality-career-nav.md" %}}

Seniors at GitLab are expected to exhibit the following competencies:

- Senior Leadership Competencies
- Senior Technical Competencies
- Senior Values Alignment

Senior Leadership Competencies

{{% include "includes/engineering-careers/senior-leadership-competency.md" %}}

Senior Technical Competencies

{{% include "includes/engineering-careers/quality-senior-technical-competency.md" %}}

Senior Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

Senior Career Matrix Worksheet

Template for Senior Career Matrix Worksheet can be found [here](#).

SECTION: engineering/careers/matrix/quality/software-engineer-in-test/staff.md

Quality Department Competencies: Staff

{{% include "includes/engineering-careers/quality-career-nav.md" %}}

Staff at GitLab are expected to exhibit the following competencies:

- Staff Leadership Competencies
- Staff Technical Competencies
- Staff Values Alignment

Staff Leadership Competencies

{{% include "includes/engineering-careers/staff-leadership-competency.md" %}}

Staff Technical Competencies

{{% include "includes/engineering-careers/quality-staff-technical-competency.md" %}}

Staff Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

Staff Career Matrix Worksheet

Template for Staff Career Matrix Worksheet can be found [here](#).

SECTION: engineering/careers/matrix/support/_index.md

Support Career Framework

{{% include "includes/engineering-careers/career-matrix-nav.md" %}}

These are the expected competencies of the Support Sub Department team members at GitLab by Job Title.

- Associate
- Intermediate
- Senior
- Staff

SECTION: engineering/careers/matrix/support/associate.md

Associate Support Engineer

{{% include "includes/engineering-careers/support-career-nav.md" %}}

Associate Support Engineers at GitLab are expected to exhibit the following competencies:

- Leadership Competencies
- Technical Competencies
- Values Alignment

Leadership Competencies

{{% include "includes/engineering-careers/associate-leadership-competency.md" %}}

Technical Competencies

- Customer Support Experience and a desire to grow Customer Support skills.
- Ability to communicate basic technical topics to customers and coworkers.
- Interest or familiarity with the following:
 - DevOps methodologies and software deployment and development process.
 - Ruby on Rails, Go, Javascript, Terraform, Ansible, Git and CI/CD.
 - Linux systems administration basics
- Learn and apply debugging techniques to resolve customer issues via email and video conferencing.
- Support a mix of Self-managed and SaaS customers.
- Create and update documentation based on customer interactions.
- Remain up to date with new product features.
- Suggest improvements to Support workflows.

Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

SECTION: engineering/careers/matrix/support/csr.md

Customer Support Representative

Leadership Competencies

{{% include "includes/engineering-careers/csr-competency.md" %}}

Technical Competencies

- Customer Support Experience and a desire to grow Customer Support skills.
- Ability to communicate basic workflows to customers and coworkers.
- Interest in or familiarity with the following:
 - Assisting with non product related licensing issues (expiry, lost licence, lost activation code).
 - Assisting field team with setting up customer contacts for trial support.
 - Assisting customer with contact management set up and maintenance.
 - Understanding the internal system architecture to help unblock data flow to support systems.
 - Assisting customers with storage and compute minute requests.
 - Assisting customers with authentication requests.
 - Managing GDPR requests.
- Learn and apply troubleshooting techniques to resolve customer issues via email and video conferencing.
- Support a mix of Self-managed and SaaS customers.
- Create and update documentation based on customer interactions.
- Remain up to date with relevant new product features and workflows.
- Suggest improvement to Support workflows.

Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

SECTION: engineering/careers/matrix/support/intermediate.md

Intermediate Support Engineer

{{% include "includes/engineering-careers/support-career-nav.md" %}}

Intermediate Support Engineers at GitLab are expected to exhibit the following competencies:

- Leadership Competencies
- Technical Competencies
- Values Alignment

Leadership Competencies

{{% include "includes/engineering-careers/intermediate-leadership-competency.md" %}}

- Demonstrates a personal growth mindset by initiating pairings, completing learning modules, and building product and troubleshooting knowledge.
- Understands support KPIs and optimizes personal workflow towards team achievement.
- Understands and adopts all documented support practices, processes and policies, and proposes appropriate changes to improve them.

Technical Competencies

{{% include "includes/engineering-careers/intermediate-technical-competency.md" %}}

- Is capable of creating and delivering clear technical replies to customer queries in spoken and written English.
- Uses strong communication techniques to understand customer situations, manage customer expectations, produce high quality ticket resolutions and create a consistently excellent customer experience.
- Takes an evidence-based approach to troubleshooting; forms a hypothesis and looks at available data to prove or disprove it.
- Knows where erroneous behaviors are likely to be logged and uses standard tooling efficiently to observe application behaviors.
- Is able to trace application behavior through GitLab codebase.
- Self-assigns issues according to the Working on Tickets Flowchart.

Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

SECTION: engineering/careers/matrix/support/senior.md

Senior Support Engineer

{{% include "includes/engineering-careers/support-career-nav.md" %}}

Senior Support Engineers at GitLab are expected to exhibit the following competencies:

- Leadership Competencies
- Technical Competencies
- Values Alignment

Leadership Competencies

{{% include "includes/engineering-careers/senior-leadership-competency.md" %}}

- Assists others with difficult issues
- Engages with other teams to resolve issues
- Mentors teammates on debugging techniques and GitLab internals
- Disseminates their expertise to the team via deep dives, group pairings, learning modules, and/or videos
- Involved with hiring new Support Engineers

Technical Competencies

{{% include "includes/engineering-careers/senior-technical-competency.md" %}}

- Able and willing to debug and resolve complex problems
- Notices and escalates emerging issues
- Becomes involved with issues that are likely to breach due to their complexity
- Contributes improvements to documentation
- Fixes customer-impacting bugs
- Strong understanding of GitLab's architecture
- Contributes to the support team toolbox

Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

SECTION: engineering/careers/matrix/support/staff.md

Support Staff Support Engineer

{{% include "includes/engineering-careers/support-career-nav.md" %}}

Staff Support Engineers at GitLab are expected to exhibit the following competencies:

- Leadership Competencies
- Technical Competencies
- Values Alignment

Leadership Competencies

{{% include "includes/engineering-careers/staff-leadership-competency.md" %}}

- Works frequently with other teams to resolve difficult issues
- Anticipates and plans for future issues that will impact Support
- Participates in Working Groups as the need arises

Technical Competencies

{{% include "includes/engineering-careers/staff-technical-competency.md" %}}

- Able to dig into the GitLab codebase and determine the root cause of complex problems
- Frequently submits fixes for customer-impacting issues
- Creates tooling to improve the efficacy of GitLab Support

Values Alignment

{{% include "includes/engineering-careers/values-competency.md" %}}

SECTION: engineering/data-engineering/analytics/monitor/_index.md

Groups

This stage consists of the following group:

- Analytics Instrumentation
- Optimize
- Platform Insights

Product Categories

These groups map to the Monitor Stage product category.

Demo Environments

In order to make it more efficient to verify changes and demonstrate our product features to customers and other stakeholders. The engineers in this stage maintain a few demo environments.

Use Case	URL
Customer simulation environment	tanuki-inc
Verifying features in Staging	monitor-sandbox (Staging)

|---|---|

| Customer simulation environment | tanuki-inc |

| Verifying features in Staging | monitor-sandbox (Staging) |

SECTION: engineering/data-engineering/analytics/monitor/optimize.md

Monitor:Optimize

Optimize FY26 direction and goals

How we work

- In accordance with our GitLab values.
- Transparently: nearly everything is public, we record/livestream meetings whenever possible.
- We get a chance to work on the things we want to work on.
- Everyone can contribute; no silos.
 - The goal is to have product give engineering and design the opportunity to be involved with direction and issue definition from the very beginning.
- We do an asynchronous daily stand-up in our group stand-up channel:
 - g-optimize-engineers-standup
- You can reach out to the team on slack at: gplanoptimize
 - All Optimize team members are encouraged to triage and respond to requests in the team Slack channel, regardless of the nature of the questions being asked.

Prioritization

Our priorities should follow overall guidance for Product. This should be reflected in the priority label for scheduled issues:

Priority	Description	Probability of shipping in milestone
priority::1	Urgent: top priority for achieving in the given milestone. These issues are the most important goals for a release and should be worked on first; some may be time-critical or unblock dependencies.	~100%
priority::2	High: important issues that have significant positive impact to the business or technical debt. Important, but not time-critical or blocking others.	~75%
priority::3	Normal: incremental improvements to existing features. These are important iterations, but deemed non-critical.	~50%
priority::4	Low: stretch issues that are acceptable to postpone into a future release.	~25%

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
```

```
  {{< tableau/filters "GROUPLABEL"="optimize" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}  
  {{< tableau/filters "GROUPLABEL"="optimize" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}  
  {{< tableau/filters "GROUPNAME"="optimize" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
```

```
{{< tableau/filters "GROUPLABEL"="optimize" >}}
{{< /tableau >}}
```

As a general guideline, we try to plan each release in this way:

- Bugs: 25%
- Features: 50%
- Maintenance: 25%

These targets will be reviewed monthly after each release during the retrospective.

SSoT for data flows across Optimize features

Data flow for Contribution analytics

Data flow for Group contribution analytics & Group value stream dashboard contributions

mermaid

flowchart TB

subgraph User Actions

useractions["User Actions\n(MRs, Pushes, Issues, etc..)"]

end

subgraph Postgres

eventstablepg["Events Table"]

end

subgraph ClickHouse

eventstablech["Events Table"]

contributionsmv["Contributions\nMaterialized View"]

end

subgraph GraphQL

contributionanalytics["ContributionAnalyticsContribution\n(Group Contribution Analytics)"]

valuestream["ValueStreamDashboardCount"]

end

useractions -->|EventCreateService| eventstablepg

eventstablepg -->|EventSyncStrategy\nevery 3min| eventstablech

eventstablech --> contributionsmv

eventstablepg --> contributionanalytics

contributionsmv --> contributionanalytics

contributionsmv --> valuestream

Data flow for AI Impact Analytics

Data flow for Group/Project AI Impact Analytics

```

mermaid
flowchart TB
    subgraph Postgres
        aicodesuggestionevents
    end
    subgraph ClickHouse
        codesuggestionusages
        duochatevents
        duochatdailyeventsmv
        codesuggestiondailyusagesmv
    end
    subgraph GraphQL
        aiMetrics
        aiUserMetrics
        aiUsageData
    end
    subgraph Redis
        Ai::UsageEventWriteBuffer
        ClickHouse::WriteBuffer
    end
    userIDE-->|/usagedata/trackevents|RESTAPI
    RESTAPI-->Ai::UsageEventWriteBuffer
    RESTAPI-->ClickHouse::WriteBuffer
    Ai::UsageEventWriteBuffer-->|sync every 5min|aicodesuggestionevents
    ClickHouse::WriteBuffer-->|sync every 5min|codesuggestionusages
    ClickHouse::WriteBuffer-->|sync every 5min|duochatevents
    duochatevents --> duochatdailyeventsmv
    codesuggestionusages --> codesuggestiondailyusagesmv
    aicodesuggestionevents --> aiUsageData
    duochatdailyeventsmv --> aiMetrics
    codesuggestiondailyusagesmv --> aiMetrics
    codesuggestiondailyusagesmv --> aiUserMetrics

```

Organizing the work

We generally follow the Product Development Flow:

1. workflow::problem validation - needs clarity on the problem to solve
1. workflow::design - needs a clear proposal (and mockups for any visual aspects)
1. workflow::solution validation - needs refinement and acceptance from engineering
1. workflow::planning breakdown - needs a Weight estimate
1. workflow::scheduling - needs a milestone assignment
1. workflow::ready for development
1. workflow::in dev
1. workflow::in review
1. workflow::verification - code is in production and pending verification by the DRI engineer
1. workflow::complete - the code has been verified and the work is complete, issue should be closed

Generally speaking, issues are in one of two states:

- Discovery/refinement: we're still answering questions that prevent us from starting development,
- Implementation: an issue is waiting for an engineer to work on it, or is actively being built.

Basecamp thinks about these stages in relation to the climb and descent of a hill.

While individual groups are free to use as many stages in the Product Development Flow workflow as they find useful, we should be somewhat prescriptive on how issues transition from discovery/refinement to implementation.

Measuring the value of the team deliverables

To visualize our flow of value to customers, we're dogfooding Value Stream Analytics to measure the time it takes to go from planning to production.

Backlog management

Backlog management is very challenging, but we try to do so with the use of labels and milestones.

Refinement

The end goal is defined, where all direct stakeholders says "yes, this is ready for development". Some issues get there quickly, some require a few passes back and forth to figure out.

The goal is for engineers to have buy-in and feel connected to the roadmap. By having engineering included earlier on, the process can be much more natural and smooth.

To do so, engineering managers, engineers, and designers can be pinged directly from the issue. We're currently exploring converting the Manage project into a group to be able to create groups to more easily ping group members.

To find issues that require refinement, please see the Next Up label and its purpose.

Next Up

- To identify issues that need refinement, use the "Next Up" label.
 - The purpose of the "Next Up" label is to identify issues that are currently in any workflow stage before workflow::ready for development. By using this "Next Up" label in addition to workflow labels, we're able to see exactly what is being refined, e.g., problem, design, solution. This helps identify which issues are closer to being ready to schedule.
 - Issues shouldn't receive a milestone for a specific release (e.g. 13.0) until they've received a · from both Product and Engineering. This also means the issue should not be labeled as workflow::ready for development.
 - Product approval is represented by an issue moving into workflow::planning breakdown.
 - Engineering approval is represented by an issue weight measuring its complexity.

Breaking down or promoting issues

Depending on the complexity of an issue, it may be necessary to break down or promote issues. A couple sample scenarios may be:

- We need to do discovery on the design, before we do anything else. A "Discovery:" issue may work best here as it helps to contain the design thinking and discussion there, with the end result being transferred over to a "Implementation:" issue. These prefixes also help to organize what type of issue they are, in the case they are linked to parent issues or epics.
- The scope of work is larger than anticipated, and needs to be broken down further, e.g., it currently has a weight higher than 5. It may suit you to then promote said issue to an epic, to break it down into smaller issues to list out the different iterations or phases of work that need to happen to deliver the overall feature that was originally proposed.
- The scope of work is clear, but a bit unwieldy for one issue. It may make sense to keep the given issue as is, to keep the conversation and activity visible to everyone, but create separate child design, backend, or frontend issues to track the more nuanced progress of a given issue.

If none of the above applies, then the issue is probably fine as-is! It's likely then that the weight of this issue is quite low, e.g., 1-2.

Managing discussions, information, decisions, and action items in an issue

As part of breaking down or promoting issues, you may find that there are a significant number of threads and comments in a given issue.

It's very important that we make sure any proposal details, pending action items, and decisions are easily visible to any stakeholder coming into an issue. Therefore, it's paramount that the issue description is kept up-to-date, or otherwise broken down or promoted as per the above section.

Estimation

Before work can begin on an issue, we should estimate it first after a preliminary investigation.

If the scope of work of a given issue touches several disciplines (docs, design, frontend, backend, etc.) and involves significant complexity across them, consider creating separate issues for each discipline (see an example).

Issues without a weight should be assigned the "workflow::planning breakdown" label.

When estimating development work, please assign an issue an appropriate weight:

| Weight | Description (Engineering) |

| ----- | ----- |

| 1 | The simplest possible change. We are confident there will be no side effects. |

| 2 | A simple change (minimal code changes), where we understand all of the requirements. |

| 3 | A simple change, but the code footprint is bigger (e.g. lots of different files, or tests affected). The requirements are clear. |

5	A more complex change that will impact multiple areas of the codebase, there may also be some refactoring involved. Requirements are understood but you feel there are likely to be some gaps along the way. We should challenge ourselves to break this issue in to smaller pieces.
8	A complex change, that will involve much of the codebase or will require lots of input from others to determine the requirements. These issues will often need further investigation or discovery before being ~workflow::ready for development and we will likely benefit from multiple, smaller issues.
13	A significant change that may have dependencies (other teams or third-parties) and we likely still don't understand all of the requirements. It's unlikely we would commit to this in a milestone, and the preference would be to further clarify requirements and/or break in to smaller Issues.

As part of estimation, if you feel the issue is in an appropriate state for an engineer to start working on it, please add the ~"workflow::ready for development" label. Alternatively, if there are still requirements to be defined or questions to be answered that you feel an engineer won't be able to easily resolve, please add the ~"workflow::blocked" label. Issues with the workflow::blocked label will appear in their own column on our planning board, making it clear that they need further attention. When applying the workflow::blocked label, please make sure to leave a comment and ping the DRI on the blocked issue and/or link the blocking issue to raise visibility.

Implementation Approach

For engineers, you may want to create an implementation approach when moving an issue out of ~workflow::planning breakdown. A proposed implementation approach isn't required to be followed, but is helpful to justify a recorded weight.

As the DRI for workflow::planning breakdown, consider following the example below to signal the end of your watch and the issues preparedness to move into scheduling. While more straightforward issues that have already been broken down may use a shorter format, the plan should (at a minimum) always justify the "why" behind an estimation.

The following is an example of an implementation approach from <https://gitlab.com/gitlab-org/gitlab/-/issues/247900implementation-plan>. It illustrates that the issue should likely be broken down into smaller sub-issues for each part of the work:

md

Implementation approach

~database

1. Add new mergerequestsauthorapproval column to namespacesettings table (The final table is TBD)

~"feature flag"

1. Create new groupmergerequestapproversrules flag for everything to live behind

~backend

1. Add new field to ee/app/services/ee/groups/updateservice.rb:117
1. Update ee/app/services/ee/namespaceettings/updateservice.rb to support more than just one setting
1. (if feature flag enabled) Update the Projects::CreateService and Groups::CreateService to update newly created projects and sub-groups with the main groups setting
1. (if feature flag enabled) Update the Groups API to show the settings value
1. Tests tests and more tests :muscle:
1. Create a seed script to generate data

~frontend

1. (if feature flag enabled) Add new Merge request approvals section to Groups general settings
1. Create new Vue app to render the contents of the section
1. Create new setting and submission process to save the value
1. Tests tests and more tests :muscle:
1. Update storybook stories for new and existing components

The DRI is highly recommended to ping a relevant counterpart or domain expert if an issue covers multiple disciplines (for example backend and frontend) before moving the issue to workflow::scheduling. This gives the domain expert the opportunity to approve the implementation plan or raise any potential pitfalls or concerns before work begins.

Once an issue has been estimated, it can then be moved to workflow::scheduling to be assigned a milestone before finally being workflow::ready for development.

Planning

We plan in monthly cycles in accordance with our Product Development Timeline. Meeting this timeline is up to the discretion of individual groups. A typical Optimize planning cycle looks like:

- By the 4th, Product should have created a planning issue for their group in the Manage project for the coming release.
 - This issue should include a tentative plan for the release, along with links to boards that represent the proposed work for the milestone.
 - A board filtered by the Next 1 - 3 releases milestone is used to capture upcoming issues.
 - Issues of particular significance to our stage's strategy should be marked with direction.
- By the 12th, all issues proposed for the next release should be estimated with weights assigned by engineering (workflow:ready for development).
 - To assist with capacity planning, we start with a capacity of 10 weight per engineer and deduct based on time off, team days, on-call schedules, or other activities. The EM captures the expected capacity in the planning issue.
 - Issues that we know will slip from the previous release should be reweighted for the remaining effort left and rescheduled to the next release.
- By the 15th, Product and Engineering will have ordered the list of issues in the Next 1 - 3 releases board.

- Depending on availability, either Product or Engineering will take capacity into consideration and assign the top issues in each type category to the next release.
- The engineering manager will assign the ~Deliverable label to any committed work.
- The entire planning process is asynchronous, however a synchronous meeting to review the final release scope is optional if Product and Engineering require additional collaboration.

Deliverable and Stretch issues

Issues labeled Deliverable are scheduled for the current milestone. They are considered top priority and are expected to be done in time for the release.

Issues labeled Stretch are stretch goals for delivering in the current milestone. If these issues are not done in the current release, they will strongly be considered for the next release.

Community contributions

Issues that have previously been agreed upon and labeled as Community contribution should be triaged to ensure they have:

- A clear implementation plan.
- A relevant weight estimate.
- The Seeking community contributors label assigned.

Once triaged the issue can be added to the backlog and left unassigned. Assigning an issue signals that the assignee is actively working on the issue, given the time constraints and varying levels of familiarity with the code base community members may have, it's best to assign the issue once an MR is progress.

If there is a clear need for the issue to be handled sooner, consider scheduling the issue for a milestone with the appropriate priority label assigned so that an optimize team member can plan for it.

If a community member expresses interest in taking on an issue, a relevant Optimize team member should ensure the issue description and implementation plan are accurate and reflect the latest decisions and all labels are up to date, as well as monitor progress in case the contributor requires additional assistance or has not been able to continue.

Self assignment

During planning, the EM may assign issues to individual engineers when it makes sense, but in general the issues will remain unassigned and left for engineers to self assign once planning has been finalized.

Expectations by role:

- EM to ping engineers on the planning issue once finalized.
- Engineers to self assign issues before the start of the release.
- EM to highlight unassigned issues during weekly team call.

During a release

- When an issue is introduced into a release after Kickoff, an equal amount of weight must be removed to account for the unplanned work.
- Development should not begin on an issue before it's been estimated and given a weight.
- By the 15th, engineering merge requests should be merged. In other words, we assume code merged after the 15th will not be in the release. That allows time for the release to be finalized, and any associated Release Posts to be merged by the 17th. (This is an experiment starting with 13.11.)

Release posts

For issues which need to be announced in more detail, a release post can be automatically created using the issue.

When working on an issue, either in planning, or during design and development, you can use the release post item generator

to have the release post created and notify all the relevant people.

If you do not want an issue to have a release post, make sure that the issue does not have a release notes section or use a release post item:: label.

Proof-of-concept MRs

We strongly believe in iteration and delivering value in small pieces. Iteration can be hard, especially when you lack product context or are working in a particularly risky/complex part of the codebase. If you are struggling to estimate an issue or determine whether it is feasible, it may be appropriate to first create a proof-of-concept MR. The goal of a proof-of-concept MR is to remove any major assumptions during planning and provide early feedback, therefore reducing risk from any future implementation.

- Create an MR, prefixed with PoC::.
- Explain what problem the PoC MR is trying to solve for in the MR description.
- Timebox it. Can you determine feasibility or a plan in less than 2-3 days?
- Identify a reviewer to provide feedback at the end of this period.
- Close the MR. Provide a summary in the original issue on what you learned from the PoC, including product and performance implications.
 - State whether you are able to move forwards with implementation or not.
 - Please do not close the issue.

The need for a proof-of-concept MR may signal that parts of our codebase or product have become overly complex. It's always worth discussing the MR as part of the retrospective so we can discuss how to avoid this step in future.

Issue triage

We generally follow the Issue Triage guidelines.

Expectations by role:

- PM is the DRI for type::feature

- EM is the DRI for type::bug
- UX supports the decision around severity labels for issues with UX, Deferred UX, and SUS
 - Where the UX severity and PM/EM severity is different, we take the higher severity of the two.
- Engineers are encouraged to participate

On a weekly basis, we aim to triage as many issues as possible. We strive to perform a complete triage on issues requiring triage.

Working on unscheduled issues

Everyone at GitLab has the freedom to manage their work as they see fit, because we measure results, not hours. Part of this is the opportunity to work on items that aren't scheduled as part of the regular monthly release. This is mostly a reiteration of items elsewhere in the handbook, and it is here to make those explicit:

1. We expect people to be managers of one, and we use GitLab ourselves. If you see something that you think is important, you can request for it to be scheduled, or you can work on a proposal yourself, as long as you keep your other priorities in mind.
1. From time to time, there are events that GitLab team-members can participate in, like the issue bash. Anyone is welcome to participate in these.

When you pick something to work on, please:

1. Follow the standard workflow and assign it to yourself.
1. Share it in gplanoptimize to encourage transparency

Additional considerations

Capacity planning

During planning we don't plan 100% of the team's capacity to go into deliverable work each milestone. Instead, we reserve a buffer of 15% per team member to allow for more time to research and scope work.

Documentation

Documentation is a crucial part of our definition of done. For any change that requires technical writing, we will add the documentation label. The documentation label should be used in addition to backend/frontend labels. If a feature justifies separate backend and frontend issues, the documentation label should be applied to each issue if applicable. An issue may only get resolved if all the work has been merged, i.e., the technical part and the documentation change.

Data seeding scripts

Features within the Optimize scope require appropriate data in order to verify functionality and test during development. Data seeding scripts should be created and/or updated as part of our development process.

Considerations for data seeding scripts:

- Ensure scripts are parameterized allowing specification of group or project ID where relevant
- Ensure scripts can be run repeatedly without failure

Feature Flags

We use feature flags as needed to ensure we provide an enterprise-level user experience to our customers. We avoid unnecessary feature flags and ensure that when introducing one, its objective is clear and we ensure the rollout dependencies and timeline stay updated. We strive to minimize long-living feature flags whenever possible and communicate changes.

The following roles and responsibilities are associated with feature flags we own:

- DRI assignment
 - The author introducing a feature flag is the DRI of the feature flag rollout.
- Auditing and cleanup
 - The EM is DRI for auditing feature flags owned within the stage and will schedule cleanups in collaboration with the feature flag DRI.
- Process improvements
 - Everyone is encouraged to contribute toward process improvements.

Meetings

Although we have a bias for asynchronous communication, synchronous meetings are necessary and should adhere to our communication guidelines. Some regular meetings that take place in Manage are:

Frequency	Meeting	DRI	Possible topics
Weekly	Group-level meeting	Engineering Managers	Ensure current release is on track by walking the board, unblock specific issues
Monthly	Planning meetings	Product Managers	See Planning section

For one-off, topic specific meetings, please always consider recording these calls and sharing them (or taking notes in a publicly available document).

Agenda documents and recordings can be placed in the shared Google drive (internal only) as a single source of truth.

Meetings that are not 1:1s or covering confidential topics should be added to the Manage Shared calendar.

All meetings should have an agenda prepared at least 12 hours in advance. If this is not the case, you are not obligated to attend the meeting. Consider meetings canceled if they do not have an agenda by the start time of the meeting.

Group Members

The following people are permanent members of the group:

```
{{< engineering/stable-counterparts role="Plan.+Optimize" >}}
```

Links and resources {links}

- Milestone retrospectives
- Our Slack channels
 - Plan:Optimize gplanoptimize
 - Daily standups g-optimize-engineers-standup
- Issue boards
 - Optimize build board and refinement board
- For more information about the optimize group's plans and vision visit the Groups page

```
## SECTION: engineering/data-engineering/analytics/monitor/platform-insights.md
```

Who we are?

The Platform Insights group is part of the GitLab Monitor stage and builds GitLab Observability and Product Analytics products.

Team members

```
{{< team-by-manager-role "Engineering Manager(.)Monitor:Platform Insights" >}}
```

Stable counterparts

```
{{< engineering/stable-counterparts manager-role="Engineering Manager(.)Monitor:Platform Insights" role="Product(.)Platform Insights|Principal(.)Monitor|Security(.)Monitor" >}}
```

Technical Architecture

Architecture Blueprints

Error Tracking

Tracing

Metrics

Logs

Architecture Documentation

See this page

Project Links

Top-level Epic
Source Code Repository

ClickHouse Datastore

Observability and analytics features have big data and insert heavy requirements which are not a good fit for Postgres or Redis. ClickHouse was selected as a good fit to meet these features requirements. ClickHouse is an open-source column-oriented database management system. It is attractive for these use cases because it can efficiently filter, aggregate, and sum across large numbers of rows. ClickHouse is not intended to replace Postgres or Redis in GitLab's stack.

We initially managed our own self-hosted Clickhouse instance, but decided to migrate to Clickhouse Cloud to enable the team to move quicker by offloading maintenance and scalability to Clickhouse.

Learn more: Clickhouse Datastore Working Group

How we work?

We base our workflow on the company's Product Development Flow. Any modifications or clarifications on how we apply the workflow are detailed below.

Async Standups

We have slack-based standups (using Geekbot) on Wednesdays and retrospectives on Fridays. We use these async standups to communicate what we have accomplished, any current blockers and what we plan to work on next.

Async Updates

Every Friday, the EM provides an async update of the team's progress, following the Ops sub-department async updates process.

These updates are published as issues in the general project.

Updates and highlights from all teams in Ops are collected automatically here, grouped by week / month / quarter.

Meetings

Weekly Team Sync: These are focused on organizing ongoing work or specific efforts such as rollout-outs or bigger initiatives.

Bi-monthly social hour: This meeting is non-work related and helps team socialize and get to know each other better.

Team member coffee chats: Each team member should schedule a coffee chat with all other team members rough every 4-6 weeks. Feel free to discuss work or non-work topics. If timezones are an issue find another way to connect, such as a async slack thread to checkin. The goal is to

get to know your other team members on a 1:1 basis.

Dev Syncs: These are developer-organized sync meetings where ICs can meet and discuss technical issues or organize technical work amongst themselves without requiring the presence of a EM.

Communication

We use several Slack channels to organize ourselves:

Primary channel: gmonitorplatforminsights

Standup channel: gmonitorplatforminsightsstandup

Social channel: gmonitorplatforminsightsinternal

How we do planning?

We are following the monthly milestone cadence. Work is organized into epics and assigned to the relevant milestones.

Milestone starting date is defined in gitlab.org group milestones. It changes every month, according to the new GitLab release calendar.

Milestone Planning timeline:

10 days before milestone starting date: Planning draft issue is created by PM/EM, with high level milestone goals.

8 days before milestone starting date: Planning draft is shared with team. Individual contributors recommend epics and issues related to these goals or carried over from previous milestones.

5 days before milestone starting date: Planning is reviewed during team sync meeting.

On milestone starting date: Milestone goals and related epics and issues should be finalized and prioritized. All planned work can be seen on the Milestone Board Previous milestone issues are moved to the new milestone or backlog.

During the milestone, we analyze progress and reprioritize as needed.

Issue prioritization

Our priorities should follow overall guidance for Product. This should be reflected in the priority label for scheduled issues:

| Priority | Description | Probability of shipping in milestone |

| ----- | ----- | ----- |

| priority::1 | Urgent: top priority for achieving in the given milestone. These issues are the most important goals for a release and should be worked on first; some may be time-critical or unblock dependencies. | ~100% |

| priority::2 | High: important issues that have significant positive impact to the business or technical debt. Important, but not time-critical or blocking others. | ~75% |

| priority::3 | Normal: incremental improvements to existing features. These are important iterations, but deemed non-critical. | ~50% |

| priority::4 | Low: stretch issues that are acceptable to postpone into a future release. | ~25% |

How to find something to work on?

Normally at the beginning of the Milestone the EM will discuss an overview of the work and what relevant areas you will focus on. Sometimes issues will already be assigned to you before the Milestone begins.

If you are ever looking for additional issues to work on:

1. Look at the Platform Insight Milestone board
1. Identify an issue that is unassigned.
1. Assign yourself to the issue.
1. Add a workflow:in dev label to the issue
1. If the scope or description are unclear, connect with the EM and or PM for clarification or (if feeling confident) groom the issue yourself and proceed.
1. Begin working on the issue.
1. Once all relevant MRs are merged, set the ~workflow::verification label.
Ensure any MRs do not auto-close issues. (Use Relates to 11111 rather than Closes 11111 in MR descriptions.)
1. Verify the changes and comment on the issue which environment you used for verification, for example Verified on production.
1. Close the issue!
1. Repeat.

How to enable Observability Beta for a customer?

To enable access to Logs, Tracing, and Metrics Beta for a certain customer, follow this process:

For SaaS:

Before hand, make sure you have the right access and permissions to run ChatOps command as detailed in this page.

Ask customer for their top-level group name (example: gitlab-org for <https://gitlab.com/gitlab-org/>)

In production, run the following commands to enable the feature flags for this group (replace gitlab-org by the customer's group name):

text

```
/chatops run feature set --group=gitlab-org observabilityfeatures true
```

To see the list of groups that have been already enabled, you can run the following command:

text

```
/chatops run feature get observabilityfeatures
```

The list returns group IDs and not group names though. To know a group's ID, browse to the group's page (example), open the "... " menu on the top-right of the page and select "Copy group ID". If you don't have access to the group, ask the customer to do it.

Learn more: see related feature flag issue.

For Self-Managed:

not available for now

Dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="platform insights" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="platform insights" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="platform insights" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="platform insights" >}}
{{< /tableau >}}
```

SECTION: engineering/data-engineering/analytics/monitor/analytics-instrumentation/_index.md

Vision

The Analytics Instrumentation Group is part of the Analytics section. Our group focuses on providing GitLab's team with data-driven product insights to build a better GitLab.

To do this, we build data collection and analytics tools within the GitLab product in a privacy-focused manner.

Insights generated from Analytics Instrumentation enable us to identify the best places to invest people and resources, what product categories mature faster, where our user experience can be improved, and how product changes impact the business.

You can learn more about what we're building next on the Analytics Instrumentation Direction page.

How we work

- We work in accordance with our GitLab values.
- We work transparently with nearly everything public.

- We get a chance to work on the things we want to work on.
- We have a bias for action.
- We make data-driven decisions.
- Everyone can contribute to our work.

How to reach us

If you have any questions start by @ mentioning the product manager for the Analytics Instrumentation Group or by creating an issue in our issue board.

Responsibilities

Internal Events

We provide the tooling to allow for instrumentation of product usage events across GitLab's services through a system we call internal events which uses the Snowplow event collection infrastructure under the hood.

A list of all instrumented events can be found in our metrics dictionary.

It is the responsibility of each engineering group to create and maintain their own events as those are domain specific, but Analytics Instrumentation is always willing to help on those issues be it via pair programming or our office hours.

We are actively monitoring the overall health of internal events via Monte Carlo alerts and Tableau dashboards.

Service Ping Metrics

We're responsible to deliver a reliable Service Ping that runs every week on SaaS and Self Managed instances. Our responsibility is tooling and automations for metric collections to set the company up for success to deliver Service Ping data to our data warehouse.

It is the responsibility of each engineering group to create and maintain their own metrics as those are domain specific, but Analytics Instrumentation is always willing to help on those issues be it via pair programming or our office hours.

A list of all metrics can be found on metrics.gitlab.com.

For questions related to a specific metric, its definition and/or implementation, please contact the Product Manager of the group which owns the metric. You can find information about the metric including its data category, whether it is considered an xMAU metric, its group designation and more in the metric dictionary.

We are actively monitoring the overall health of Service Ping via Monte Carlo alerts and Tableau dashboards.

Projects

These are important projects that we maintain besides our involvement in the main GitLab codebase.

1. Version App

1. Metrics Dictionary and Metrics Dictionary from Growth group.

1. Snowplow Pseudonymization

1. Iglu Schema repository for GitLab

Team members

The following people are permanent members of the Analytics Instrumentation Group:

```
{{< engineering/stable-counterparts role="Analytics.+Analytics.Instrumentation" >}}
```

Project management process

Our team uses a hybrid of Scrum for our project management process. This process follows GitLab's monthly milestone release cycle.

- We only work off of issue boards which act as our single source of truth.
- We continuously progress issues to the next workflow stage.
- We work on both product and engineering initiatives.
- We prioritize and estimate all issues we work on.
- We do monthly milestone planning to prepare for our upcoming milestone.
- We do weekly check-ins to share updates within the team

Check-In

We do a weekly automated check-in within a separate Slack channel.

We feel that a weekly cadence is enough to keep everyone up to date about the most important developments within the team.

A bot asks every team member automatically the following questions at the beginning of the week and posts them to the channel

- What Victories did you have in the last week?: This should be the most important achievements from the last week. They can also be personal achievements.
- What Impediments did you have in the last week? Which impediments do you anticipate this week?: This is important information for other stakeholders, to understand where a team member might need support.
- Which Priorities do you have for the current week?: This should be the most important things you want to achieve during the week.

Retrospective

We believe that continuously evolving and improving the way we work is crucial to delivering good outcomes. Traditionally, retrospectives are used to facilitate this improvement.

Due to our geographical distribution we cannot run typical sync retrospectives that include everyone.

We've also found that completely async retrospectives only see limited participation and discussion.

We're currently trialing a mixed retrospective format, we're team members pair on filling in the retro issue. The process is as follows:

1. Every milestone the async-retrospective project automatically creates an issue in the analytics-instrumentation retro project which contains a list of retro

questions.

1. Every 4th week a donut Slackbot will pair up all team members in random groups of two.

1. Every group of two should pair on filling out the retro issue for the current milestone together within that week:

1. This should happen ideally through a sync meeting or alternatively via Slack.

1. Focus on the what happened the last two weeks, since the last pairing.

1. Pairing should involve taking the time to think through the questions, talking about the answers with your pair and coming up with action items based on the identified problems.

1. Expect an overall time commitment of 30 minutes to an hour for the bi-weekly pairing.

1. Every 2nd week we have a 30-minute time blocker event, that is meant for everyone to take the time to answer Retro discussions.

Workflow

Our team use the following workflow stages defined in the Product Development Flow:

Validation stage

| Label | Usage |

| -- | -- |

| ~"workflow::validation backlog" | Applied by the Product Manager for incoming issues that have not been refined or prioritized. |

| ~"workflow::problem validation" | Applied by the Product Manager for issues where the PM is developing a thorough understanding of the problem |

| ~"workflow::design" | Applied by the Product Manager or Designer (or Analytics Instrumentation Engineer) to ideate and propose solutions. The proposed solutions should be reviewed by engineering to ensure technical feasibility. |

| ~"workflow::solution validation" | Applied by the Product Manager or Designer (or Analytics Instrumentation Engineer) to validate a proposed solution through user interviews or usability testing. |

Build stage

| Label | Usage |

| -- | -- | -- |

| ~"workflow::planning breakdown" | Applied by the Product Manager for Engineers to begin breaking down issues and adding estimates. |

| ~"workflow::ready for development" | Applied by either Engineering or Product Manager after an issue has been broken down and scheduled for development. |

| ~"workflow::in dev" | Applied by the Engineer after work (including documentation) has begun on the issue. An MR is typically linked to the issue at some point throughout this stage. |

| ~"workflow::in review" | Applied by the Engineer indicating that all MRs required to close an issue are in review. |

| ~"workflow::verification" | Applied by the Engineer after the MRs in the issue have been merged, this label is applied signaling the issue needs to be verified in staging or production. |

| ~"workflow::complete" | Applied by the Engineer after all MRs have merged and the issue has been verified. At this step, the issue should also be closed. |

| ~"workflow::blocked" | Applied by any team member if at any time during development the issue is blocked. For example: technical issue, open question to PM or PD, cross-group dependency. |

OKRs & Epics

Analytics Instrumentation uses OKRs to plan quarterly goals and direct prioritization. To make key results present in daily prioritization we create an epic for every objective or key result, like in the FY25Q2 OKRs.

Our long term roadmap is written down on our direction page.

Issue boards

We use issue boards to track issue progress on a daily basis. Issue boards are our single source of truth for the status of our work. Issue boards should be viewed at the highest group level for visibility into all nested projects in a group.

- Analytics Instrumentation Issue Board - Current Milestone
- Analytics Instrumentation Issue Board - by milestone

Picking something to work on

Engineers can find and open the board for the current milestone.

Engineers should start at the top of the "workflow::ready for development" column and pick the first available